

# ***Trabajo Práctico 1***

## ***- Autómatas -***

| <i>Grupo 1</i>                     |                  |
|------------------------------------|------------------|
| <i>Integrantes</i>                 | <i>Legajo</i>    |
| <i>Nicolás Orlando Vázquez</i>     | <i>203.327-6</i> |
| <i>Yair César Minaya Gutiérrez</i> | <i>222.525-6</i> |
| <i>Gonzalo Ezequiel Raposo</i>     | <i>218.363-1</i> |
| <i>Lautaro Mendoza Aguirre</i>     | <i>222.517-7</i> |

*Profesora: Ing.Roxana Leituz*



Código Fuente:

Comenzamos con la implementación formal del AFD mediante una Matriz. Esta Estructura modela la tabla de transiciones donde cada fila representa un estado del autómata

```
1  #include <stdio.h>
2
3  int TT[8][8] = {
4      /* 0 */ { 1, 3, 2, 2, 7, 7, 0, 7},
5      /* 1 */ { 7, 2, 2, 2, 7, 7, 7, 7},
6      /* 2 */ { 7, 2, 2, 2, 7, 7, 0, 7},
7      /* 3 */ { 7, 6, 6, 7, 4, 7, 0, 7},
8      /* 4 */ { 7, 5, 5, 5, 7, 5, 7, 7},
9      /* 5 */ { 7, 5, 5, 5, 7, 5, 0, 7},
10     /* 6 */ { 7, 6, 6, 7, 7, 7, 0, 7},
11     /* 7 */ { 7, 7, 7, 7, 7, 7, 0, 7},
12 };
```

Luego, continuamos declarando la función auxiliar columna, la cual traduce el flujo de caracteres ASCII a los índices de la columna de la matriz. Es decir, recibe un carácter y devuelve un entero del 1 al 7, el cual servirá para recorrer nuestra matriz

```
int columna(char c) {
    if (c == '+' || c == '-') {
        return 0;
    } else if (c == '0') {
        return 1;
    } else if (c >= '1' && c <= '7') {
        return 2;
    } else if (c >= '8' && c <= '9') {
        return 3;
    } else if (c == 'x' || c == 'X') {
        return 4;
    } else if (c >= 'a' && c <= 'f' || c >= 'A' && c <= 'F') {
        return 5;
    } else if (c == '@') {
        return 6;
    } else {
        return 7;
    }
}
```

Continuamos con la función auxiliar analizarCadena.

Para comenzar, definimos las variables destinadas al recuento de los grupos de números y las fallas léxicas requeridas. Asimismo, establecemos los valores iniciales para estado\_actual y estado\_anterior, indispensables para registrar la transición dentro del AFD.

Luego, abrimos un ciclo while que se encarga de recorrer la cadena ingresada carácter por carácter, deteniéndose únicamente al encontrar el carácter nulo \0 que marca el final del texto. Dentro de este ciclo, tomamos el carácter en la posición actual y se lo pasamos a nuestra función auxiliar columna para saber qué índice de la matriz debemos mirar.

Antes de realizar la transición, guardamos una copia de nuestra posición en la variable estado\_anterior. Inmediatamente después, actualizamos nuestro estado\_actual consultando la matriz TT con la fila y columna correspondientes.

Cuando el carácter @ es leído, la matriz está configurada para volver al estado inicial 0-, por lo que empleamos el valor guardado en estado\_anterior a fin de analizar el lexema finalizado. En caso de que dicha posición previa coincida con una condición final válida (es decir, 2 si es decimal, 3 o 6 en el caso de octal, o 5 para hexadecimal), sumamos una unidad en la cuenta respectiva. De lo contrario, si finalizó en la posición de falla o se interrumpió la lectura, contabilizamos una falla de tipo léxico.

Por último, resulta necesario considerar que la constante final finaliza con el carácter de fin de texto `\0` en lugar del delimitador `@`. En consecuencia, una vez concluida la iteración del `while`, incorporamos una verificación equivalente. La diferencia radica en que se analiza de manera directa el `estado_actual` en el que permaneció el AFD para registrar dicho último valor. Concluyendo la ejecución, mostramos en consola el total computado en cada acumulador.

```
void analizarCadena(char* cadena) {
    int estado_actual = 0;
    int estado_anterior = 0;
    int cant_decimales = 0, cant_octales = 0, cant_hexadecimales = 0, errores = 0;

    int i = 0;
    while (cadena[i] != '\0') {
        char c = cadena[i];
        int col = columna(c);

        estado_anterior = estado_actual;
        estado_actual = TT[estado_actual][col];

        if (c == '@') {
            if (estado_anterior == 2) cant_decimales++;
            else if (estado_anterior == 3 || estado_anterior == 6) cant_octales++;
            else if (estado_anterior == 5) cant_hexadecimales++;
            else if (estado_anterior != 0) {
                errores++;
                printf("Error lexico detectado antes de la posicion %d\n", i);
            }
        }
        i++;
    }

    if (estado_actual == 2) cant_decimales++;
    else if (estado_actual == 3 || estado_actual == 6) cant_octales++;
    else if (estado_actual == 5) cant_hexadecimales++;
    else if (estado_actual != 0) {
        errores++;
        printf("Error lexico al final de la cadena principal\n");
    }

    printf("\n--- RESULTADOS EJERCICIO 1 ---\n");
    printf("Decimales: %d\n", cant_decimales);
    printf("Octales: %d\n", cant_octales);
    printf("Hexadecimales: %d\n", cant_hexadecimales);
    printf("Errores lexicos: %d\n", errores);
}
```

Para finalizar, hardcodeamos una cadena de prueba en el `main` y llamamos a la función `analizarCadena`

```
int main() {
    char cadenaPrueba[] = "123@-45@012@0x1A@89a@+78@0XF1";
    printf("Analizando cadena: %s\n", cadenaPrueba);
    analizarCadena(cadenaPrueba);

    return 0;
}
```

```
PS C:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\02-TP1-AUTOMATAS> .\ejercicio1.exe
Analizando cadena: 123@-45@012@0x1A@89a@+78@0XF1
Error lexico detectado antes de la posicion 20

--- RESULTADOS EJERCICIO 1 ---
Decimales: 3
Octales: 1
Hexadecimales: 2
Errores lexicos: 1
```

## Ejercicio 2

Debe realizar una función que reciba un carácter numérico y retorne un número entero.

Como los caracteres se almacenan utilizando su valor en la tabla ASCII, y los dígitos del '0' al '9' están ordenados de forma consecutiva (del 48 al 57), la conversión se puede hacer con una resta básica.

Si la función recibe el carácter '5', su valor interno en ASCII es 53

El carácter '0' tiene el valor ASCII 48

Al restarlos (53 - 48), la computadora te devuelve exactamente el entero 5

```
int caracterAEnteroSeguro(char c) {  
    if (c >= '0' && c <= '9') {  
        return c - '0';  
    }  
    return -1; // Retorna un código de error si el carácter no es numérico  
}
```

```
int main() {  
  
    printf("\n--- RESULTADOS EJERCICIO 2 ---\n");  
    char charNum = '7';  
    int numConvertido = caracterAEnteroSeguro(charNum);  
    printf("El caracter '%c' convertido a entero es: %d\n", charNum, numConvertido);  
    return 0;  
}
```

```
--- RESULTADOS EJERCICIO 2 ---  
El caracter '7' convertido a entero es: 7
```

Ejercicio 3:

*Ingresar una cadena que represente una operación simple con enteros decimales y obtener su resultado, se debe operar con +, - y \*. Ejemplo =  $3+4*7+3-5 = 29$ . Considerando que el alfabeto son los números decimales, signos +, - y \* valide con un autómata previo a la operación que dicha cadena que representa la operación pertenezca al lenguaje.*

Nuestro alfabeto será:

$\Sigma = \{ \text{Numeros, +, -, *} \}$

El autómata debe garantizar que la cadena empiece con un número, alterne con operadores y termine siempre en un número.

Por lo tanto:

**0-** : Espera un dígito para ir a q1.

**1+** : Leyó un número. Si lee otro dígito, sigue en q1. Si lee un operador (+, -, \*), va a 2.

**2 (Operador)**: Leyó un operador. Espera obligatoriamente un dígito para volver a 1.

**3 (Error)**: Cualquier otro carácter o secuencia inválida (ej. dos operadores seguidos).

Definición Formal del Autómata:  $M = \{Q, \Sigma, T, q_0, F\}$

Conjunto de Estados de Aceptación:  $F = \{q_1\}$

### Definición de Columnas

Dígito: 0 al 9.

Operador: +, -, \*.

Fin de cadena: \0.

Otro: Cualquier carácter inválido (letras, espacios, otros símbolos).

### Tabla de Transiciones:

| Estado     | Dígito | Operador | Fin      | Otro |
|------------|--------|----------|----------|------|
| <b>0</b>   | 1      | 3        | 3        | 3    |
| <b>1 +</b> | 1      | 2        | <b>1</b> | 3    |
| <b>2</b>   | 1      | 3        | 3        | 3    |
| <b>3</b>   | 3      | 3        | 3        | 3    |

## Código Fuente:

Para abordar el tercer ejercicio, el enunciado nos exige validar explícitamente que la cadena matemática ingresada pertenezca al lenguaje antes de intentar resolverla. Para cumplir con esto, implementamos un Automata Finito Determinístico (AFD) mediante la matriz TT\_Ej3 y su respectiva función de mapeo columna\_Ej3 (La cual recibe un carácter y devuelve un entero del 1 al 7, el cual servirá para recorrer nuestra matriz). También agregamos una función auxiliar que nos convierte los números recibidos en ASCII a Enteros.

```
1  #include <stdio.h>
2  #include <ctype.h>
3
4  // --- FUNCIONES AUXILIARES PREVIAS ---
5  int caracterAEntero(char c) {
6      if (c >= '0' && c <= '9') {
7          return c - '0';
8      }
9      return -1;
10 }
11
12 // --- 1. MATRIZ Y VALIDACIÓN (EL AFD) ---
13 // Columnas: 0: Dígito, 1: Operador, 2: Fin (\0), 3: Otro
14 int TT_Ej3[4][4] = {
15     /* 0 */ { 1, 3, 3, 3 },
16     /* 1 */ { 1, 2, 9, 3 }, // '9' representa el estado de aceptación final
17     /* 2 */ { 1, 3, 3, 3 },
18     /* 3 */ { 3, 3, 3, 3 } // Estado de error
19 };
20
21 int columnaEj3(char c) {
22     if (isdigit(c)) return 0;
23     if (c == '+' || c == '-' || c == '*') return 1;
24     if (c == '\0') return 2;
25     return 3;
26 }
27
```

A diferencia del primer ejercicio, aquí nuestro alfabeto se reduce a cuatro grupos lógicos: dígitos, operadores aritméticos (+, -, \*), el fin de cadena (\0), y cualquier otro carácter inválido. La matriz está diseñada para forzar una alternancia estricta: si el autómata lee un dígito, avanza al estado 1; si lee un operador, avanza al estado 2 y queda a la espera obligatoria de otro dígito.

El proceso de verificación se efectúa a través del método esOperacionValida. Dicha rutina examina el texto evaluando transiciones en la matriz. De encontrar un orden incorrecto (como dos signos continuos del tipo ++ o un signo al cierre), el sistema deriva al estado de error 3 devolviendo 0 para desestimar el ingreso. Exclusivamente si la secuencia concluye de forma adecuada en un número y procesa el centinela \0, la máquina pasa a la posición final 9 confirmando la validez con un 1.

```
✓ int esOperacionValida(char* cadena) {
    int estado_actual = 0;
    int i = 0;
    while (1) {
        char c = cadena[i];
        int col = columnaEj3(c);
        estado_actual = TT_Ej3[estado_actual][col];
        if (estado_actual == 9) return 1; // Aceptada formalmente por el AFD
        if (estado_actual == 3) return 0; // Rechazada por error léxico/sintáctico
        i++;
    }
}
```

Una vez el automata detecta que el la cadena es valida, procedemos a realizar el cálculo en la función calcularOperacion.

Para resolverlo, optamos por implementar la lógica de una 'Pila de Operandos' utilizando un arreglo estático (pila[100]). El algoritmo funciona de la siguiente manera: A medida que recorremos la cadena, vamos construyendo los números carácter por carácter auxiliándonos con la función caracterAEntero. Cuando nos topamos con un operador, evaluamos el operador que habíamos guardado en el paso anterior (inicializado por defecto en +).

Si el operador previo era un +, simplemente 'apilamos' el número actual.

Si era un -, apilamos el número pero invertido a negativo.

Si el operador previo era un \* (multiplicación), no creamos un nuevo elemento en la pila. En su lugar, tomamos el valor que se encontraba en el tope de nuestra pila, lo multiplicamos por el número actual, y reemplazamos el tope con este nuevo resultado.

De esta forma, las multiplicaciones se resuelven en el momento y las sumas y restas quedan pendientes en la pila. Al finalizar de recorrer la cadena entera, el cálculo final se reduce a iterar sobre nuestra pila y sumar todos los valores almacenados.

```
44
45 // --- 2. EVALUACIÓN Y PRECEDENCIA ---
46 void calcularOperacion(char* cadena) {
47     // Primero, el autómata valida que la cadena pertenezca al lenguaje
48     if (!esOperacionValida(cadena)) {
49         printf("Error: La operacion '%s' no es valida o no pertenece al lenguaje.\n", cadena);
50         return;
51     }
52
53     int pila[100];
54     int tope = -1;
55     char operador_previo = '+';
56     int num_actual = 0;
57
58     int i = 0;
59     while (cadena[i] != '\0') {
60         char c = cadena[i];
61
62         // Armos el número si tiene más de un dígito
63         if (isdigit(c)) {
64             num_actual = (num_actual * 10) + caracterAEntero(c);
65         }
66
67         // Si encontramos un operador o llegamos al último dígito de la cadena
68         if ((!isdigit(c) && c != ' ') || cadena[i+1] == '\0') {
69             if (operador_previo == '+') {
70                 pila[++tope] = num_actual;
71             } else if (operador_previo == '-') {
72                 pila[++tope] = -num_actual; // Apilamos el negativo
73             } else if (operador_previo == '*') {
74                 // Resolución de precedencia: sacamos el anterior, multiplicamos y devolvemos a la pila
75                 pila[tope] = pila[tope] * num_actual;
76             }
77             operador_previo = c;
78             num_actual = 0;
79         }
80         i++;
81     }
82
83     // Sumamos todos los valores que quedaron en la pila
84     int resultado = 0;
85     for (int j = 0; j <= tope; j++) {
86         resultado += pila[j];
87     }
88
89     printf("El resultado de '%s' es: %d\n", cadena, resultado);
90 }
91
```

Para validar la robustez de nuestra solución, implementamos una batería de pruebas en la función main. Como se observa en la consola de ejecución, el algoritmo resolvió con éxito las operaciones válidas, respetando perfectamente la precedencia de operadores.

A su vez, sometimos al validador a tres casos de error deliberados: una cadena con dos operadores consecutivos (3++4), una que iniciaba con un operador (\*5+2) y otra que finalizaba en un operador (4+5-). En los tres casos, nuestro Autómata Finito interceptó la anomalía léxico/sintáctica antes de realizar cualquier cálculo, emitiendo el mensaje de rechazo correspondiente y demostrando la fiabilidad del diseño propuesto.

```
91
92 // --- BLOQUE PRINCIPAL PARA PROBAR ---
93 int main() {
94     printf("--- PRUEBAS EJERCICIO 3 ---\n");
95
96     // Caso de éxito del enunciado
97     char op1[] = "3+4*7+3-5";
98     calcularOperacion(op1);
99
100    // Otros casos de éxito
101    char op2[] = "10*2-5*3+1";
102    calcularOperacion(op2);
103
104    // Casos de falla (deben ser atajados por el autómata)
105    char opError1[] = "3++4"; // Dos operadores seguidos
106    calcularOperacion(opError1);
107
108    char opError2[] = "*5+2"; // Empieza con operador
109    calcularOperacion(opError2);
110
111    char opError3[] = "4+5-"; // Termina con operador
112    calcularOperacion(opError3);
113
114    return 0;
115 }
```

```
PS C:\Users\NoxiePC\Desktop\SintaxisDeParadigmas\02-TP1-AUTOMATAS> .\ejercicio3.exe
--- PRUEBAS EJERCICIO 3 ---
El resultado de '3+4*7+3-5' es: 29
El resultado de '10*2-5*3+1' es: 6
Error: La operacion '3++4' no es valida o no pertenece al lenguaje.
Error: La operacion '*5+2' no es valida o no pertenece al lenguaje.
Error: La operacion '4+5-' no es valida o no pertenece al lenguaje.
```